A Practical Comparative Study of Scalable and Secure Backend Architectures for Resource-Constrained Web Applications

line 1: 1st Given Name Surname
line 2: *dept. name of organization (of Affiliation)*
line 3: *name of organization (of Affiliation)*
line 4: City, Country
line 5: email address or ORCID

line 1: 2nd Given Name Surname
line 2: *dept. name of organization (of Affiliation)*
line 3: *name of organization (of Affiliation)*
line 4: City, Country
line 5: email address or ORCID

line 1: 3rd Given Name Surname
line 2: *dept. name of organization (of Affiliation)*
line 3: *name of organization (of Affiliation)*
line 4: City, Country
line 5: email address or ORCID

line 1: 4th Given Name Surname
line 2: *dept. name of organization (of Affiliation)*
line 3: *name of organization (of Affiliation)*
line 4: City, Country
line 5: email address or ORCID

line 1: 5th Given Name Surname
line 2: *dept. name of organization (of Affiliation)*
line 3: *name of organization (of Affiliation)*
line 4: City, Country
line 5: email address or ORCID

line 1: 6th Given Name Surname
line 2: *dept. name of organization (of Affiliation)*
line 3: *name of organization (of Affiliation)*
line 4: City, Country
line 5: email address or ORCID

***Abstract*— Modern web applications face the challenge of handling continuously growing user traffic while maintaining adequate security levels under resource constraints such as limited budgets and computational capacity. Selecting the appropriate backend architecture remains non-trivial, particularly for small development teams, research institutions, and startups. This paper presents an empirical comparison of three widely used backend architectures—monolithic, microservices, and hybrid—in terms of scalability and security for resource-constrained web applications. Experimental results demonstrate that the monolithic architecture performs adequately under low workloads but degrades significantly at high concurrency. The microservices architecture sustains the highest throughput under load but incurs greater memory overhead. The hybrid architecture provides a strong balance between performance and security by centralizing authentication and rate limiting at the gateway layer.**

Keywords— Backend Architecture, Scalability, Security, Microservices, Resource-Constrained Systems

I. Introduction

The proliferation of web-based services has placed increasing demands on backend infrastructure. Developers must select an architecture that can scale under variable traffic while maintaining security, all within practical budget and hardware constraints. This decision is especially consequential for small organizations such as startups and research institutions, where over-engineering incurs unacceptable cost and under-engineering leads to failures under load.

Three architectural patterns dominate modern backend development: the monolithic architecture, in which all application components reside in a single process; the microservices architecture, in which components are independently deployed and communicate over a network; and the hybrid architecture, which places an API gateway in front of a monolithic core to offload cross-cutting concerns such as authentication and rate limiting.

Prior work has compared monolithic and microservices architectures in terms of throughput and resource consumption [1][2], but few studies examine these patterns under the specific constraints of shared commodity hardware with limited memory. This paper addresses that gap through a controlled experiment on two consumer-grade machines, comparing the three architectures across response time, throughput, memory consumption, and security behavior under low, medium, and high load.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 describes the methodology and experimental setup. Section 4 presents experimental results. Section 5 discusses limitations. Section 6 concludes.

II. Related Work

organizational editing before formatting. Please note sections A-D below for more information on proofreading, spelling and grammar.

Keep your text and graphic files separate until after the text has been formatted and styled. Do not use hard tabs, and limit use of hard returns to only one return at the end of a paragraph. Do not add any kind of pagination anywhere in the paper. Do not number text heads-the template will do that for you.

[1] On a single machine, the monolith outperforms its microservice equivalent, and vertical scaling is more cost-efficient than horizontal scaling in the Azure cloud environment, which is relevant to web applications with tight computational budgets.

[2] Empirical concurrency testing showed that monolithic architecture exhibits approximately 6% higher throughput than microservices, while load testing revealed no significant difference between the two, highlighting the importance of workload characteristics in architectural selection.

[3] The performance of Kubernetes, K3s, KubeEdge, and ioFog is assessed in terms of memory footprint and support for remote edge devices, providing concrete benchmarks for

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

orchestration platform selection in low-resource web backends.

[4] Containerization and Kubernetes have become the dominant approach in cloud infrastructure, with lightweight distributions such as K3s and KubeEdge emerging for edge and constrained environments.

[5] Non-parametric regression on stress test data demonstrated that microservices architecture achieves better hardware resource optimization, cost reduction, and productivity compared to monolithic systems, providing a quantitative basis for architectural selection.

[6] JSON Web Tokens based on RFC 7519 have become a widely accepted solution for stateless authentication; however, they do not incorporate user behavioral history into access control decisions. An extension is proposed that increases the trustworthiness of token-based access control by incorporating behavioral weights.

[7] An analysis of four case studies of reversion from microservices to monolithic architectures—including a widely reported cost reduction achieved by Prime Video—identifies cost, complexity, scalability, and performance as the primary motivations, offering practical guidance for resource-constrained systems.

[8] A comparison of monolithic and microservices architectures across Go, Python, and C++ on constrained IoT devices finds that the monolithic architecture is better suited for small-scale applications in terms of power consumption, performance, and memory usage.

[9] A survey of WebAssembly runtimes—including browser-integrated runtimes (Chrome, Firefox, Edge) and standalone runtimes (Wasmtime, Wasmer, WasmEdge, Wasm3)—evaluates execution modes (interpretation, JIT, AOT) and finds that runtime performance is highly dependent on the execution environment.

[10] A comparison of eight deep learning architectures for solar power forecasting finds that sequence-based methods such as LSTM and GRU perform best for temporal dependencies, with model selection depending on the trade-off between forecasting accuracy and computational cost.

[11] A Dockerized framework using Dynamic RESTful APIs and FastAPI is proposed to address the limitations of static API architectures in IoT applications. Evaluation shows high throughput with a 3.11% error rate under heavy load and low latency under varying conditions.

[12] A comparison of WebAssembly and Docker containers across startup time, execution performance, memory size, portability, and security isolation finds that while Wasm has advantages in cold start times and binary size, Docker remains more mature and flexible for complex deployments.

[13] Docker containers are demonstrated to effectively support microservices deployment by decoupling services and managing dependencies within CI/CD pipelines, with positive effects on scalability, portability, and deployment of distributed systems.

[14] Kubernetes horizontal scaling, per-node-pool cluster scaling, and automatic resource adjustment are applied to overcome scalability challenges for fog servers in edge environments.

[15] A survey of edge AI acceleration techniques—covering model compression (pruning, quantization, knowledge distillation), Neural Architecture Search, and compiler methodologies (TVM, TensorRT, OpenVINO)—concludes that integrating all three pillars reduces latency, memory footprint, and energy consumption while preserving model accuracy.

III. METHODOLOGY

A. Application

The test application is divided into three functional modules: a write-heavy data processing endpoint that inserts records and performs aggregation, a report generation endpoint that queries and returns aggregated data, and an authentication module that manages registration, login, and JWT issuance. To isolate the effect of architecture rather than implementation differences, identical business logic was maintained across all three configurations. No additional features were included; the modules are intentionally minimal.

B. Hardware and Software

Two Dell OptiPlex 7080 machines from the college laboratory were used. Both are equipped with an Intel Core i5-10400 (6 cores, 2.9 GHz), 8 GB DDR4 RAM, 256 GB SSD, and run Ubuntu 22.04 LTS. One machine hosted the application and databases; the other was dedicated to k6 load generation. The machines were connected over a wired LAN. Tests were conducted during evening hours to minimize interference from shared network activity during the day.

Software versions: Node.js 18.17.1, Express 4.18.2, PostgreSQL 15.3, MongoDB 6.0.7, Docker 24.0.5, NGINX 1.24.0, PM2 5.3.0, k6 v0.45.0. PM2 was used for process management across all three setups to ensure consistency. PM2 logging was configured with timestamps to improve readability of run logs.

C. Architecture Details

Monolith.

A single Node.js process in which all three modules operate as Express routers within the same application. PostgreSQL handles user and session data; MongoDB stores processed records and reports. PM2 was added to all three configurations after initial testing revealed that the application crashed under heavy load, requiring manual restarts. The addition of PM2 was applied uniformly to maintain comparability across architectures.

Microservices.

Each of the three functional components—authentication, data processing, and reporting—runs as an independent Node.js application in a separate Docker container. Services communicate via REST over a Docker user-defined bridge network. During initial testing, inter-container latency spikes of 5–15 ms were observed. Investigation revealed that the default Docker bridge network performs additional DNS resolution per call; switching to a user-defined bridge with static IPs resolved the issue.

Per-service database isolation—standard practice in microservices—was not fully implemented due to hardware and time constraints. A single PostgreSQL instance and a single MongoDB instance were shared across services, with separate databases and credentials per service. This design decision limits the degree of fault isolation and may have affected some results; this is discussed further in Section 5.

Hybrid.

The same monolithic Node.js application is deployed behind an NGINX reverse proxy. NGINX handles JWT authentication via the ngx_http_auth_jwt_module, rate limiting via limit_req_zone (100 requests per second per IP, returning HTTP 429 on excess), routing, and TLS termination. Requests reaching the application layer have already been validated at the gateway; the Node.js process performs no authentication itself. The other two architectures did not include TLS, as testing was conducted on a local network. This asymmetry is a limitation of the security comparison and is noted in Section 5.

D. Load Testing

Load testing was performed using k6 at three virtual user (VU) levels: 50 VUs (low), 200 VUs (medium), and 500 VUs (high). Each test ran for 3 minutes at a constant VU count, with 30-second ramp-up and ramp-down periods. Each load level was tested three times and results were averaged. The k6 script followed a realistic mixed workload: authenticate to obtain a JWT token, submit five sequential data processing requests, then submit two sequential report requests. A 10-second timeout was applied; requests that timed out or returned 5xx status codes were counted as errors.

The following metrics were collected: average response time, p95 response time, requests per second (RPS), memory RSS, CPU utilization, and HTTP error rate. CPU and memory measurements were captured every 5 seconds on the application machine using a custom monitoring script that logged output to a file for post-run analysis.

IV. Experiments and Results

A. Response Time & Throughput

Table 1 presents response time and throughput results for all three architectures at all three load levels.

Values are averages over three runs.

Architecture	Load (VUs)	Avg RT (ms)	p95 RT (ms)	RPS	Error (%)
Monolith	50	42	73	117	0.00
Microservices	50	63	101	80	0.00
Hybrid	50	55	86	91	0.00
Monolith	200	191	423	103	0.33
Microservices	200	129	207	154	0.00
Hybrid	200	146	241	137	0.00
Monolith	500	1347	3210	69	4.71
Microservices	500	188	318	265	0.19
Hybrid	500	258	503	194	0.51

Table 1. Response time (RT) and throughput at three load levels. Averages over three runs.

At 50 VUs, all three architectures performed without errors. The monolith achieved the lowest average response time (42 ms) due to the absence of inter-service network calls and gateway forwarding overhead. The microservices configuration was approximately 20 ms slower on average, attributed to REST communication between Docker containers.

At 200 VUs, the monolith began to degrade. Average response time rose to 191 ms with a p95 of 423 ms and an error rate of 0.33%. Errors were caused by PostgreSQL connection pool exhaustion: the default pool size of 10 connections was insufficient at 200 concurrent users issuing authentication queries. The pool was deliberately left at the default value to capture behavior without manual tuning. Both the microservices and hybrid configurations completed the 200 VU test without errors.

At 500 VUs, the monolith exhibited severe degradation: average response time reached 1347 ms, p95 exceeded 3 seconds, and the error rate was 4.71%. Errors were predominantly HTTP 503 responses rather than timeouts. The Node.js event loop was operating at 88–92% of a single CPU core throughout the test, consistent with the single-threaded execution model saturating under concurrent request queuing.

The microservices configuration sustained 188 ms average response time at 500 VUs with a 0.19% error rate, with errors attributable to shared MongoDB connection limits across three containers. The hybrid configuration averaged 258 ms at 500 VUs. This result was slower than microservices, likely due to NGINX and the Node.js process competing for CPU on the same machine; however, this cannot be confirmed without isolating NGINX onto separate hardware.

B. Memory and CPU

During the 500 VU test, the three microservices containers together consumed approximately 840–870 MB RSS. The monolith used approximately 410 MB and the hybrid configuration approximately 590 MB total (including NGINX).

During initial high-load microservices testing, the machine ran out of memory. Without container memory limits, the combined RSS of all containers and database processes approached 7 GB on the 8 GB machine, causing the OS to swap and rendering the system unusable. Memory limits of 512 MB per container were subsequently applied via Docker Compose. All results reported in Table 1 reflect this configuration.

CPU profiling showed the monolith sustaining approximately 90% utilization on a single core throughout the 500 VU test. Individual microservices containers each remained below 60% CPU. In the hybrid setup, the Node.js process utilized approximately 70% of one core while NGINX consumed approximately 12% on a separate core. CPU monitoring data is not presented as a figure due to formatting issues with the monitoring script output that could not be resolved prior to submission.

C. Security Behavior

Three security properties were evaluated: rejection of invalid JWT tokens, enforcement of rate limiting, and handling of repeated failed authentication attempts from the same IP address.

In the monolith, JWT validation is implemented in Express middleware applied before all protected routes. All 200 requests sent with invalid or expired tokens were correctly rejected with HTTP 401. The principal security risk in this configuration is misconfiguration: because authentication logic is centralized in the application, a single error affects all routes simultaneously.

In the microservices configuration, each service implements its own token validation. During testing, a defect was discovered in the data processing service: the service validated the JWT signature but did not check the exp (expiration) claim, allowing expired tokens to be accepted. This bug was identified through deliberate testing with a token approximately three hours past its expiry. It illustrates a systemic risk of the microservices pattern: when security logic is independently implemented across multiple services, inconsistencies are likely. The authentication and reporting services did not exhibit this defect; only the data processing service was affected.

In the hybrid configuration, all authentication logic is handled exclusively by NGINX before requests reach the application. Invalid, expired, and missing tokens all returned HTTP 401; rate-limited requests returned HTTP 429. Centralizing this logic in a single gateway component eliminates the risk of inconsistent implementation across services.

V. Limitations

Several limitations should be considered when interpreting these results. First, the microservices configuration did not implement per-service database isolation as recommended by the microservices pattern. All services shared a single PostgreSQL and a single MongoDB instance with separate databases and credentials. This likely underestimates the resource overhead of a production-grade microservices deployment and may have affected connection contention results.

Second, the hybrid configuration included TLS termination at the NGINX layer, while the monolith and microservices configurations did not. This asymmetry makes direct security comparison imprecise, though it reflects a practical deployment scenario on a local network.

Third, NGINX and the Node.js application shared the same physical machine in the hybrid setup. Co-location of the gateway and application introduces CPU contention that would not occur in a production deployment with dedicated infrastructure. The slower hybrid results at 500 VUs may partially reflect this constraint rather than inherent architectural overhead.

Fourth, the test machines are shared resources in a college laboratory. Although tests were conducted during off-peak evening hours, background processes and network conditions were not fully controlled. Results may not be fully reproducible in other environments.

Finally, no database-level tuning was applied—connection pool sizes, query caches, and index configurations were left at defaults. The connection pool exhaustion observed in the monolith at 200 VUs, for example, could be mitigated by increasing the default pool size. The intent was to compare the architectures as-deployed without manual optimization, but this may disadvantage configurations that are more sensitive to default settings.

VI. Conclusion

This paper compared monolithic, microservices, and hybrid backend architectures on commodity hardware under controlled load conditions. The experimental results support the following conclusions.

The monolithic architecture is the most performant at low load due to the absence of inter-service communication overhead, achieving 42 ms average response time at 50 VUs. However, it degrades sharply at higher concurrency: connection pool exhaustion caused errors at 200 VUs, and single-threaded event loop saturation produced a 4.71% error rate and 1347 ms average response time at 500 VUs. The monolith is appropriate only for applications that do not expect significant concurrency growth.

The microservices architecture provided the best throughput at high load (265 RPS at 500 VUs) with a low error rate (0.19%), but at the cost of approximately double the memory consumption of the monolith. The independent implementation of JWT validation across services introduced a security defect that would not have occurred in a centralized configuration, highlighting the need for shared security libraries or gateway-level enforcement in microservices deployments.

The hybrid architecture offered a practical balance: competitive throughput at medium and high load, zero errors at 200 VUs, and the strongest security posture through centralized gateway-level authentication and rate limiting. The JWT expiry defect observed in microservices would have been prevented by the hybrid approach. For resource-constrained teams requiring a path to improved scalability without full microservices complexity, the hybrid architecture represents a viable and operationally simpler option.

Future work should evaluate these architectures on isolated hardware per component, with production-grade database configurations and per-service database instances, to obtain results more representative of real deployment conditions

VII. References

[1] G. Blinowski, A. Ojdowska and A. Przybyłek, "Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation," IEEE Access, vol. 10, pp. 20357–20374, 2022, doi: 10.1109/ACCESS.2022.3152803.

[2] O. Al-Debagy and P. Martinek, "A Comparative Review of Microservices and Monolithic Architectures," IEEE 18th International Symposium on Computational Intelligence and

Informatics (CINTI), Budapest, Hungary, 2018, pp. 000149–000154, doi: 10.1109/CINTI.2018.8928192.

[3] I. Čilić, P. Krivić, I. Podnar Žarko, and M. Kušek, "Performance Evaluation of Container Orchestration Tools in Edge Computing Environments," Sensors, vol. 23, p. 4008, 2023, doi: 10.3390/s23084008.

[4] R. Vaño, I. Lacalle, P. Sowiński, R. S-Julián, and C. E. Palau, "Cloud-Native Workload Orchestration at the Edge: A Deployment Review and Future Directions," Sensors, vol. 23, p. 2215, 2023, doi: 10.3390/s23042215.

[5] F. Tapia et al., "From Monolithic Systems to Microservices: A Comparative Study of Performance," Applied Sciences, vol. 10, p. 5797, 2020, doi: 10.3390/app10175797.

[6] A. Bucko, K. Vishi, B. Krasniqi, and B. Rexha, "Enhancing JWT Authentication and Authorization in Web Applications Based on User Behavior History," Computers, vol. 12, p. 78, 2023, doi: 10.3390/computers12040078.

[7] R. Su, X. Li, and D. Taibi, "From Microservice to Monolith: A Multivocal Literature Review," Electronics, vol. 13, p. 1452, 2024, doi: 10.3390/electronics13081452.

[8] S. du Plessis and N. Correia, "A Comparative Study of Software Architectures in Constrained Device IoT Deployments," IEEE International Conference on Internet of Things and Intelligence Systems (IoTaIS), 2021.

[9] M. Ţălu, "A Comparative Study of WebAssembly Runtimes: Performance Metrics, Integration Challenges, Application Domains, and Security Features," Archives of Advanced Engineering Science, 2025.

[10] M. Abdelsattar et al., "Comparative Analysis of Deep Learning Architectures in Solar Power Prediction," Scientific Reports, vol. 15, p. 31729, 2025.

[11] E. Mabotha, N. E. Mabunda, and A. Ali, "A Dockerized Approach to Dynamic Endpoint Management for RESTful Application Programming Interfaces in Internet of Things Ecosystems," Sensors, vol. 25, p. 2993, 2025.

[12] N. A. Nguyen, "Comparison of WebAssembly and Container Technologies," 2025.

[13] M. M. Raikar et al., "Leveraging Docker Containers for Deployment of Web Applications in Microservices Architecture," First International Conference for Women in Computing (InCoWoCo), IEEE, 2024.

[14] K. Aruna and P. Gurunathan, "Enhancing Edge Environment Scalability: Leveraging Kubernetes for Container Orchestration and Optimization," Concurrency and Computation: Practice and Experience, vol. 36, p. e8303, 2024.

[15] J. Sander et al., "On Accelerating Edge AI: Optimizing Resource-Constrained Environments," arXiv preprint arXiv:2501.15014, 2025.